Day 2 학습 정리 — 2026-06-28

오늘 한 것

- `POST /children` — 프로필 6개 필드 저장 추가
- `PATCH /children/{id}` — 아이 정보 수정 엔드포인트 구현

핵심 개념

파일 역할 구분

파일	역할	비유
`schemas.py`	어떤 데이터를 받을지/줄지 약속	주문서 양식
`models.py`	DB 테이블 구조 정의	창고 선반 설계도
`children_router.py`	요청 받아서 실제로 처리	직원이 일하는 곳
`main.py`	앱 전체를 켜고 라우터 연결	회사 대표

흐름:

```
사용자 요청 → schemas.py (데이터 검증) → children_router.py (처리) → models.py (DB 저장)
```

---

main.py가 아니라 children_router.py에 쓰는 이유

나중에 라우터가 여러 개 생김:

```
main.py
  ├── children_router.py       (아이 관련)
  ├── lesson_records_router.py (수업 기록 관련)
  └── feedbacks_router.py      (피드백 관련)
```

전부 main.py에 넣으면 수백 줄이 돼서 관리 불가. 역할별로 파일을 나눠서 main.py에서 연결만 함.

POST vs PATCH

	POST	PATCH
목적	새 데이터 **생성**	기존 데이터 **일부 수정**
동작	없던 걸 만듦	있는 것 중 일부만 바꿈

Child() 생성 패턴 (POST)

```
new_child = Child(
    name=child.name,          # 왼쪽: DB 컬럼 이름 (models.py)
    memory_hint=child.memory_hint,  # 오른쪽: 사용자가 보낸 데이터 (schemas.py)
    personality=child.personality,
    ...
)
db.add(new_child)
db.commit()
db.refresh(new_child)
return new_child
```

PATCH 패턴

```
# 1. DB에서 아이 찾기
db_child = db.query(Child).filter(Child.id == child_id).first()

# 2. 보내온 데이터 중 실제로 값이 있는 것만 꺼내기
update_data = child.model_dump(exclude_unset=True)
# exclude_unset=True → 사용자가 보내지 않은 필드 제외 (None으로 덮어쓰기 방지)

# 3. DB 객체에 하나씩 반영
for key, value in update_data.items():
    setattr(db_child, key, value)  # db_child.age = 9 와 같은 말

# 4. 저장
db.commit()
db.refresh(db_child)
return db_child
```

---

model_dump() 란?

Pydantic `BaseModel`을 상속받으면 자동으로 쓸 수 있는 기능. 객체를 딕셔너리로 변환해줌.

```
child.model_dump()
# → {"name": None, "age": 9, "gender": None, ...}  # None 포함 전부

child.model_dump(exclude_unset=True)
# → {"age": 9}  # 사용자가 실제로 보낸 것만
```

setattr() 란?

`setattr(객체, 속성이름, 값)` = `객체.속성이름 = 값`

key가 문자열로 들어올 때 쓰는 방식:

```
setattr(db_child, "age", 9)
# → db_child.age = 9 와 같음
```

오늘 내가 직접 만든 코드

```
# PATCH /children/{child_id}
@router.patch("/{child_id}")
def update_child(child_id: int, child: ChildUpdate, db: Session =
Depends(get_db)):
    db_child = db.query(Child).filter(Child.id == child_id).first()
    update_data = child.model_dump(exclude_unset=True)
    for key, value in update_data.items():
        setattr(db_child, key, value)
    db.commit()
    db.refresh(db_child)
    return db_child
```